

# Arrays and Pointers - Answers

---

*Note: Code solutions are provided in the [files/svar/](#) directory. This file contains explanations, PBL answers, and references to the solution files.*

---

## Answer: Arrays 1 - Basic Array

**Solution file:** [files/svar/arrays1\\_sol.c](#) (or [arrays1\\_sol2.c](#))

**Expected Output:**

```
2
3
5
7
```

**PBL Answer:** Print values in reverse order:

```
#include <stdio.h>

int main() {
    int Numbers[] = {2, 3, 5, 7};
    int size = sizeof(Numbers) / sizeof(Numbers[0]);

    // Print in reverse
    for (int i = size - 1; i >= 0; i--) {
        printf("%d\n", Numbers[i]);
    }

    return 0;
}
```

Output: 7, 5, 3, 2

---

## Answer: Arrays 2 - Dynamic Allocation

**Solution file:** [files/opgaver/arrays2.c](#) (question file)

**Answers:**

- **What does the code do?** Creates a dynamically sized array using `malloc`, fills it with random values (0-299), picks a random element, then counts how many elements are greater than that random element.

- **What makes this dynamic memory allocation?** The array size is determined at runtime using `rand()`, not at compile time.
- **Is this heap or stack allocation?** Heap allocation - `malloc` allocates from the heap.
- **Advantages:**
  - **Heap:** Flexible size, can grow/shrink as needed, persists beyond function scope
  - **Stack:** Faster allocation, automatic deallocation, but fixed size

**PBL Answer:** If we don't free allocated memory:

- **Memory leak:** The memory remains allocated but inaccessible
- The program uses more and more memory over time
- Eventually can cause the system to run out of memory
- The OS won't reclaim it until the program terminates
- In long-running programs, this can cause crashes

---

## Answer: Pointers Basics

**Solution file:** `files/svar/pointers1_sol.c`

**Expected Output:**

```
Value of num: 42
Address of num: 0x7ffd42a1b3ac
Value pointed by p_num: 42
Address stored in p_num: 0x7ffd42a1b3ac
```

*(Addresses will vary each run)*

**PBL Answer:** Function that modifies via pointer:

```
#include <stdio.h>

void modifyValue(int *ptr) {
    *ptr = 100; // Dereference and change the value
}

int main() {
    int num = 42;
    int *p_num = &num;

    printf("Before: num = %d\n", num);

    modifyValue(p_num); // Pass pointer to function

    printf("After: num = %d\n", num); // num is now 100
}
```

```
    return 0;
}
```

---

## Answer: Descriptive Statistics 1 - Sum

**Solution file:** [files/svar/descriptive1\\_solution.c](files/svar/descriptive1_solution.c)

**Expected Output:**

```
Sum: 8928
```

**PBL Answer:** Sum of positive numbers only:

```
#include <stdio.h>

int main() {
    int arr[] = {151, 123, 7, 1, -31, 0, 17, 9921, 9, 15, -1003, -3, 17};
    int sum = 0;
    int sum_positive = 0;
    int size = sizeof(arr) / sizeof(arr[0]);

    for (int i = 0; i < size; i++) {
        sum += arr[i];
        if (arr[i] > 0) {
            sum_positive += arr[i];
        }
    }

    printf("Sum of all: %d\n", sum);
    printf("Sum of positive only: %d\n", sum_positive);

    return 0;
}
```

Output: Sum of positive only: 10124

---

## Answer: Descriptive Statistics 2 - Mean

**Solution file:** [files/svar/descriptive2\\_solution.c](files/svar/descriptive2_solution.c)

**Expected Output:**

```
Mean: 686.769231
```

**PBL Answer:** Mean of positive numbers only:

```
#include <stdio.h>

int main() {
    int arr[] = {151, 123, 7, 1, -31, 0, 17, 9921, 9, 15, -1003, -3, 17};
    int sum_positive = 0;
    int count_positive = 0;
    int size = sizeof(arr) / sizeof(arr[0]);

    for (int i = 0; i < size; i++) {
        if (arr[i] > 0) {
            sum_positive += arr[i];
            count_positive++;
        }
    }

    float mean_positive = (float)sum_positive / count_positive;
    printf("Mean of positive numbers: %.2f\n", mean_positive);

    return 0;
}
```

Output: Mean of positive numbers: 595.53

---

## Answer: Descriptive Statistics 3 - Mean and Standard Deviation

**Solution file:** `files/svar/descriptive3_solution.c`

**Expected Output:**

```
Age: mean = 46.40, std = 24.12
Height: mean = 1.71, std = 0.12
Weight: mean = 71.64, std = 13.59
```

*(Exact values depend on implementation)*

**PBL Answer:** Calculate median:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Comparison function for qsort
int compare(const void *a, const void *b) {
    float fa = *(const float *)a;
    float fb = *(const float *)b;
```

```
        return (fa > fb) - (fa < fb);
    }

float calculate_median(float arr[], int n) {
    // Make a copy to sort
    float *sorted = malloc(n * sizeof(float));
    for (int i = 0; i < n; i++) sorted[i] = arr[i];

    qsort(sorted, n, sizeof(float), compare);

    float median;
    if (n % 2 == 0) {
        median = (sorted[n/2 - 1] + sorted[n/2]) / 2.0;
    } else {
        median = sorted[n/2];
    }

    free(sorted);
    return median;
}

int main() {
    float age[] = {23, 35, 67, 42, 54, 18, 29, 72, 31, 88};
    float height[] = {1.78, 1.62, 1.50, 1.90, 1.70, 1.55, 1.82, 1.60, 1.75,
1.85};
    float weight[] = {70.5, 55.3, 60.2, 85.7, 72.4, 50.1, 68.9, 64.0, 75.3,
90.2};

    printf("Age: mean = %.2f, std = %.2f, median = %.2f\n",
        mean(age, 10), std(age, 10), calculate_median(age, 10));
    printf("Height: mean = %.2f, std = %.2f, median = %.2f\n",
        mean(height, 10), std(height, 10), calculate_median(height,
10));
    printf("Weight: mean = %.2f, std = %.2f, median = %.2f\n",
        mean(weight, 10), std(weight, 10), calculate_median(weight,
10));

    return 0;
}
```

---

## File References

All answer files are located in: [files/svar/](#) or [files/opgaver/](#)

- Arrays 1: [arrays1\\_sol.c](#) or [arrays1\\_sol2.c](#)
- Arrays 2: [opgaver/arrays2.c](#) (question file - solution is conceptual understanding)
- Pointers: [pointers1\\_sol.c](#)
- Descriptive 1 (Sum): [descriptive1\\_solution.c](#)
- Descriptive 2 (Mean): [descriptive2\\_solution.c](#)
- Descriptive 3 (Mean + Std): [descriptive3\\_solution.c](#)